Sélection de variables : une introduction

Lino Galiana
Version du 16 octobre 2026

L'accès à des bases de données de plus en plus riches permet des modélisations de plus en plus raffinées. Cependant, les modèles parcimonieux sont généralement préférables aux modèles extrêmement riches pour obtenir de bonnes performances sur un nouveau jeu de données (prédictions *out-of-sample*). Les méthodes de sélection de variables, notamment le `LASSO`, permettent de sélectionner le signal le plus pertinent dilué au milieu du bruit lorsqu'on a beaucoup d'information à traiter.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Principe du LASSO **2**
1.1 Principe général 2
1.2 Pénalisation 2
1.3 Première régression LASSO 3
2 Rôle de la pénalisation α sur la sélection de variables **7**
3 Validation croisée pour sélectionner le modèle **8**
Informations additionnelles 9

L'ensemble de la partie *machine learning* utilise le même jeu de données, présenté dans l'introduction de cette partie : les données de vote aux élections présidentielles américaines croisées à des variables sociodémographiques. Le code est disponible sur Github.

```
!pip install geopandas openpyxl plotnine plotly
```

```
import requests

url     =     'https://raw.githubusercontent.com/linogaliana/python-datascientist/main/content/
modelisation/get_data.py'
r = requests.get(url, allow_redirects=True)
open('getdata.py', 'wb').write(r.content)

import getdata
votes = getdata.create_votes_dataframes()
```

Jusqu'à présent, nous avons supposé que les variables utiles à la prévision du vote Républicain étaient connues du modélisateur. Nous n'avons ainsi exploité qu'une partie limitée des variables disponibles dans nos données. Néanmoins, outre le fléau computationnel que représenterait la construction d'un modèle avec un grand nombre de variables, le choix d'un nombre restreint de variables (modèle parcimonieux) limite le risque de sur-apprentissage.

Comment, dès lors, choisir le bon nombre de variables et la meilleure combinaison de ces variables ? Il existe de multiples méthodes, parmi lesquelles :

- se fonder sur des critères statistiques de performance qui pénalisent les modèles non parcimonieux. Par exemple, le BIC.

- techniques de *backward elimination*.
- construire des modèles pour lesquels la statistique d'intérêt pénalise l'absence de parcimonie (ce que l'on va souhaiter faire ici).

Dans ce chapitre, nous allons présenter les enjeux principaux de la sélection de variables par le biais du LASSO.

Nous allons utiliser par la suite les fonctions ou packages suivants :

```
import numpy as np
from sklearn.svm import LinearSVC
from sklearn.feature_selection import SelectFromModel
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Lasso
import sklearn.metrics
from sklearn.linear_model import LinearRegression
import matplotlib.cm as cm
import matplotlib.pyplot as plt
from sklearn.linear_model import lasso_path
import seaborn as sns
```

1 Principe du LASSO

1.1 Principe général

La classe des modèles de *feature selection* est ainsi très vaste et regroupe un ensemble très divers de modèles. Nous allons nous focaliser sur le LASSO (*Least Absolute Shrinkage and Selection Operator*) qui est une extension de la régression linéaire qui vise à sélectionner des modèles *sparses*. Ce type de modèle est central dans le champ du *Compressed sensing* (où on emploie plutôt le terme de *L1-regularization* que de LASSO). Le LASSO est un cas particulier des régressions elastic-net dont un autre cas fameux est la régression *ridge*. Contrairement à la régression linéaire classique, elles fonctionnent également dans un cadre où $p > N$, c'est-à-dire où le nombre de régresseurs est très grand puisque supérieur au nombre d'observations.

1.2 Pénalisation

En adoptant le principe d'une fonction objectif pénalisée, le LASSO permet de fixer un certain nombre de coefficients à 0. Les variables dont la norme est non nulle passent ainsi le test de sélection.

> Le programme d'optimisation du LASSO
>
> Le LASSO est un programme d'optimisation sous contrainte. On cherche à trouver l'estimateur β qui minimise l'erreur quadratique (régression linéaire) sous une contrainte additionnelle régularisant les paramètres: $$ \min_{\beta} \frac{1}{2}\mathbb{E}\bigg(\big(X\beta - y \big)^2 \bigg) \\ \text{s.t. } \sum_{j=1}^p |\beta_j| \leq t $$
>
> Ce programme se reformule grâce au Lagrangien et permet ainsi d'obtenir un programme de minimisation plus maniable :

$$\beta^{\text{LASSO}} = \arg\min_{\beta} \frac{1}{2}\mathbb{E}\left((X\beta - y)^2 \right) + \alpha \sum_{j=1}^{p} |\beta_j| = \arg\min_{\beta} ||y - X\beta||_2^2 + \lambda ||\beta||_1$$

où λ est une réécriture de la régularisation précédente qui dépend de α. La force de la pénalité appliquée aux modèles non parcimonieux dépend de ce paramètre.

1.3 Première régression LASSO

Comme nous cherchons à trouver les meilleurs prédicteurs du vote Républicain, nous allons retirer les variables qui sont dérivables directement de celles-ci : les scores des concurrents !

```
import pandas as pd

df2 = pd.DataFrame(votes.drop(columns='geometry'))
df2 = df2.loc[
  :,
  ~df2.columns.str.endswith(
    ('_democrat','_green','_other', 'winner', 'per_point_diff', 'per_dem')
    )
  ]


df2 = df2.loc[:,~df2.columns.duplicated()]
```

Dans le prochain exercice, nous allons utiliser la fonction suivante pour avoir une matrice de corrélation plus esthétique que celle permise par défaut avec `Pandas`.

```
import numpy as np
import pandas as pd
import plotly.express as px


def plot_corr_heatmap(
    df: pd.DataFrame,
    drop_cols=None,
    column_labels: dict | None = None,
    decimals: int = 2,
    width: int = 600,
    height: int = 600,
    show_xlabels: bool = False
):
    """
    Trace une heatmap de corrélation (triangle inférieur) à partir d'un DataFrame.

    Paramètres
    ----------
    df : pd.DataFrame
        DataFrame d'entrée.
    drop_cols : list ou None
        Liste de colonnes à supprimer avant le calcul de la corrélation
        (ex: ['winner']).
    column_labels : dict ou None
        Dictionnaire pour renommer les colonnes (ex: column_labels).
    decimals : int
        Nombre de décimales pour l'arrondi avant corr().
```

```
    width, height : int
        Dimensions de la figure Plotly.
    show_xlabels : bool
        Afficher ou non les labels en abscisse.
    """
    data = df.copy()

    # 1. Colonnes à drop
    if drop_cols is not None:
        data = data.drop(columns=drop_cols)

    # 2. Arrondi + renommage éventuel
    if column_labels is not None:
        data = data.rename(columns=column_labels)
    data = data.round(decimals)

    # 3. Matrice de corrélation
    corr = data.corr()

    # 4. Masque triangle supérieur
    mask = np.triu(np.ones_like(corr, dtype=bool))
    corr_masked = corr.mask(mask)

    # 5. Heatmap Plotly
    fig = px.imshow(
        corr_masked.values,
        x=corr.columns,
        y=corr.columns,
        color_continuous_scale='RdBu_r',  # échelle inversée
        zmin=-1,
        zmax=1,
        text_auto=".2f"
    )

    # 6. Hover custom
    fig.update_traces(
        hovertemplate="Var 1: %{y}<br>Var 2: %{x}<br>Corr: %{z:.2f}<extra></extra>"
    )

    # 7. Layout
    fig.update_layout(
        coloraxis_showscale=False,
        xaxis=dict(
            showticklabels=show_xlabels,
            title=None,
            ticks=''
        ),
        yaxis=dict(
            showticklabels=show_xlabels,
            title=None,
            ticks=''
        ),
        plot_bgcolor="rgba(0,0,0,0)",
        margin=dict(t=10, b=10, l=10, r=10),
        width=width,
        height=height
    )
```

```
    return fig
```

Dans cet exercice, nous utiliserons également une fonction pour extraire les variables sélectionnées par le LASSO, la voici :

```
from sklearn.linear_model import Lasso
from sklearn.pipeline import Pipeline

def extract_features_selected(lasso: Pipeline, preprocessing_step_name: str = 'preprocess') -
> pd.Series:
    """
    Extracts selected features based on the coefficients obtained from Lasso regression.

    Parameters:
    - lasso (Pipeline): The scikit-learn pipeline containing a trained Lasso regression model.
    - preprocessing_step_name (str): The name of the preprocessing step in the pipeline. Default
is 'preprocess'.

    Returns:
    - pd.Series: A Pandas Series containing selected features with non-zero coefficients.
    """
    # Check if lasso object is provided
    if not isinstance(lasso, Pipeline):
        raise ValueError("The provided lasso object is not a scikit-learn pipeline.")

    # Extract the final transformer from the pipeline
    lasso_model = lasso[-1]

    # Check if lasso_model is a Lasso regression model
    if not isinstance(lasso_model, Lasso):
        raise ValueError("The final step of the pipeline is not a Lasso regression model.")

    # Check if lasso model has 'coef_' attribute
    if not hasattr(lasso_model, 'coef_'):
       raise ValueError("The provided Lasso regression model does not have 'coef_' attribute. "
                        "Make sure it is a trained Lasso regression model.")

    # Get feature names from the preprocessing step
    features_preprocessing = lasso[preprocessing_step_name].get_feature_names_out()

    # Extract selected features based on non-zero coefficients
    features_selec = pd.Series(features_preprocessing[np.abs(lasso_model.coef_) > 0])

    return features_selec
```

Exercice 1 : Premier LASSO

On cherche toujours à prédire la variable `per_gop`. Avant de faire notre estimation, nous allons créer certains objets intermédiaires qui seront utilisés pour définir notre *pipeline*:

1. Dans notre `DataFrame`, remplacer les valeurs infinies par des `NaN`.
2. Créez un échantillon d'entraînement et un échantillon test.

Maintenant nous pouvons passer au coeur de la définition de notre *pipeline*. Cet exemple pourra servir de source d'inspiration, ainsi que celui-ci.

3. Créer en premier lieu les étapes de *preprocessing* pour notre modèle. Pour cela, il est d'usage de séparer les étapes appliquées aux variables numériques continues des autres variables, dites catégorielles.

- Pour les **variables numériques**, imputer à la moyenne puis effectuer une standardisation ;
- Pour les **variables catégorielles**, les techniques de régression linéaires impliquent d'utiliser une expansion par *one hot encoding*. Avant de faire ce *one hot encoding*, faire une imputation par valeur la plus fréquente.

4. Finaliser le *pipeline* en ajoutant l'étape d'estimation puis estimer un modèle LASSO pénalisé avec $\alpha = 0.1$.

En supposant que votre *pipeline* soit dans un objet nommé `pipeline` et que la dernière étape est nommée `model`, vous pouvez directement accéder à cette étape en utilisant l'objet `pipeline['model']`.

5. Afficher les valeurs des coefficients. Quelles variables ont une valeur non nulle ?
6. Montrer que les variables sélectionnées sont parfois très corrélées.
7. Comparer la performance de ce modèle parcimonieux avec celle d'un modèle avec plus de variables.

Aide pour la question 1

```
# Remplacer les infinis par des NaN
df2.replace([np.inf, -np.inf], np.nan, inplace=True)
```

Aide pour la question 3 La définition d'un pipeline suit la structure suivante :

```
numeric_pipeline = Pipeline(steps=[
    ('impute', # définir la méthode d'imputation ici
     ),
    ('scale', # définir la méthode de standardisation ici
    )
])

categorical_pipeline = # adapter le template

# À vous de définir en amont numerical_features et categorical_features
preprocessor = ColumnTransformer(transformers=[
    ('number', numeric_pipeline, numerical_features),
    ('category', categorical_pipeline, categorical_features)
])
```

Le *pipeline* de *preprocessing* (question 3) prend la forme suivante :

Le *pipeline* prend la forme suivante, une fois finalisé (question 4) :

À l'issue de la question 5, les variables sélectionnées sont :

Le modèle est assez parcimonieux puisqu'un sous-échantillon de nos variables initiales (d'autant que nos variables catégorielles ont été éclatées en de nombreuses variables par le *one hot encoding*).

Certaines variables font sens, comme les variables d'éducation par exemple. Notamment, un des meilleurs prédicteurs pour le score des Républicains en 2020 est… le score des Républicains (et mécaniquement des démocrates) en 2016 et 2012.

Par ailleurs, on sélectionne des variables redondantes. Une phase plus approfondie de nettoyage des données serait en réalité nécessaire.

Le modèle parcimonieux est (légèrement) plus performant :

D'ailleurs, on pourrait déjà remarquer que régresser le score de 2020 sur celui de 2016 amène déjà à de très bonnes performances explicatives, ce qui suggère que le vote se comporte comme un processus autorégressif :

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
smf.ols("per_gop ~ share_2016_REPUBLICAN", data = df2).fit().summary()
```

2 Rôle de la pénalisation α sur la sélection de variables

Nous avons jusqu'à présent pris l'hyperparamètre α comme donné. Quel rôle joue-t-il dans les conclusions de notre modélisation ? Pour cela, nous pouvons explorer l'effet que sa valeur peut avoir sur le nombre de variables passant l'étape de sélection.

Pour le prochain exercice, nous allons considérer exclusivement les variables quantitatives pour accélérer les calculs. En effet, avec des modèles non parcimonieux, les multiples modalités de nos variables catégorielles rendent le problème d'optimisation difficilement tractable.

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

df2.replace([np.inf, -np.inf], np.nan, inplace=True)
X_train, X_test, y_train, y_test = train_test_split(
    df2.drop(["per_gop"], axis = 1),
    100*df2['per_gop'], test_size=0.2, random_state=0
)

numerical_features = X_train.select_dtypes(include='number').columns.tolist()
categorical_features = X_train.select_dtypes(exclude='number').columns.tolist()

numeric_pipeline = Pipeline(steps=[
    ('impute', SimpleImputer(strategy='mean')),
    ('scale', StandardScaler())
])

preprocessed_features = pd.DataFrame(
  numeric_pipeline.fit_transform(
    X_train.drop(columns = categorical_features)
  )
)

numeric_pipeline
```

> Exercice 2 : Rôle du paramètre de pénalisation
>
> Utiliser la fonction `lasso_path` pour évaluer le nombre de paramètres sélectionnés par LASSO lorsque α varie (parcourir $\alpha \in [0.001, 0.01, 0.02, 0.025, 0.05, 0.1, 0.25, 0.5, 0.8, 1.0]$).

La relation que vous devriez obtenir entre α et le nombre de paramètres est celle-ci :

On voit que plus α est élevé, moins le modèle sélectionne de variables.

3 Validation croisée pour sélectionner le modèle

Quel α faut-il privilégier ? Pour cela, il convient d'effectuer une validation croisée afin de choisir le modèle pour lequel les variables qui passent la phase de sélection permettent de mieux prédire le résultat Républicain.

```
from sklearn.linear_model import LassoCV

my_alphas = np.array([0.001,0.01,0.02,0.025,0.05,0.1,0.25,0.5,0.8,1.0])

lcv = (
  LassoCV(
    alphas=my_alphas,
    fit_intercept=False,
    random_state=0,
    cv=5
    ).fit(
      preprocessed_features, y_train
    )
)
```

On peut récupérer le *« meilleur »* α :

```
print("alpha optimal :", lcv.alpha_)
```

Celui-ci peut être utilisé pour faire tourner un nouveau *pipeline* :

```
from sklearn.compose import make_column_transformer, ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_pipeline = Pipeline(steps=[
    ('impute', SimpleImputer(strategy='mean')),
    ('scale', StandardScaler())
])

categorical_pipeline = Pipeline(steps=[
    ('impute', SimpleImputer(strategy='most_frequent')),
    ('one-hot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer(transformers=[
    ('number', numeric_pipeline, numerical_features),
    ('category', categorical_pipeline, categorical_features)
])

model = Lasso(
  fit_intercept=False,
  alpha = lcv.alpha_
)

lasso_pipeline = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('model', model)
])
```

```
lasso_optimal = lasso_pipeline.fit(X_train,y_train)

features_selec2 = extract_features_selected(lasso_optimal)
```

Les variables sélectionnées sont :

> **Tip**
>
> Dans le cas où le modèle paraîtrait trop peu parcimonieux, il faudrait revoir la phase de définition des variables pertinentes pour comprendre si des échelles différentes de certaines variables ne seraient pas plus appropriées (par exemple du `log`).

Informations additionnelles

> **Environnement Python**
>
> Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).
>
> L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
```

```
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.